

汇智海洋——设备检修知识检索与作业系统·部署及运维文档

文档版本：V1.0 | 编制日期：2026-06-18 | 适用软件版本：V0.1.0+

编制单位：汇智海洋项目组

目录

- 1 系统架构概述
 - 1.1 系统组成
 - 1.2 部署架构图
 - 1.3 容器编排说明
- 2 环境要求与基础依赖安装
 - 2.1 硬件与操作系统要求
 - 2.2 Docker 与 Docker Compose 安装
 - 2.3 配置龙芯 Docker 镜像源 (LoongArch 必需)
 - 2.4 获取项目源码
- 3 项目配置文件准备
 - 3.1 后端环境变量 (.env)
 - 3.2 大模型 API 配置 (数据库方式)
 - 3.3 前端构建配置
- 4 使用 Docker Compose 一键部署
 - 4.1 部署前检查清单
 - 4.2 构建镜像
 - 4.3 启动服务
 - 4.4 验证部署
 - 4.5 首次初始化操作
- 5 手动分步部署 (不使用 Docker)
 - 5.1 Python 环境安装
 - 5.2 Node.js 环境安装
 - 5.3 后端部署
 - 5.4 前端部署
 - 5.5 Nginx 配置
- 6 运维管理
 - 6.1 常用 Docker 命令
 - 6.2 查看日志
 - 6.3 重启与更新
 - 6.4 数据备份
 - 6.5 健康监控
- 7 故障排除
 - 7.1 常见启动问题
 - 7.2 LoongArch 特定问题
 - 7.3 数据库锁定问题

- 7.4 AI 服务不可用
 - 7.5 端口冲突
- 8 附录
 - 8.1 目录结构
 - 8.2 端口清单
 - 8.3 Docker 命令速查表
 - 8.4 龙芯架构适配说明

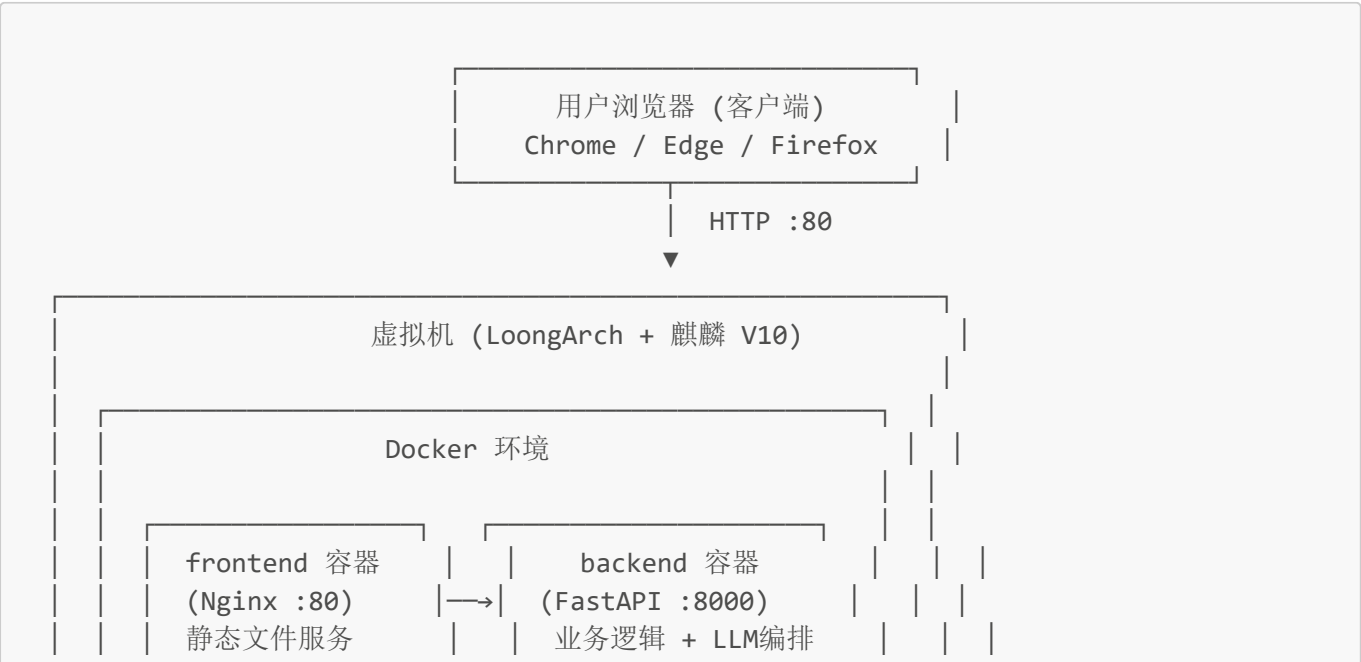
1 系统架构概述

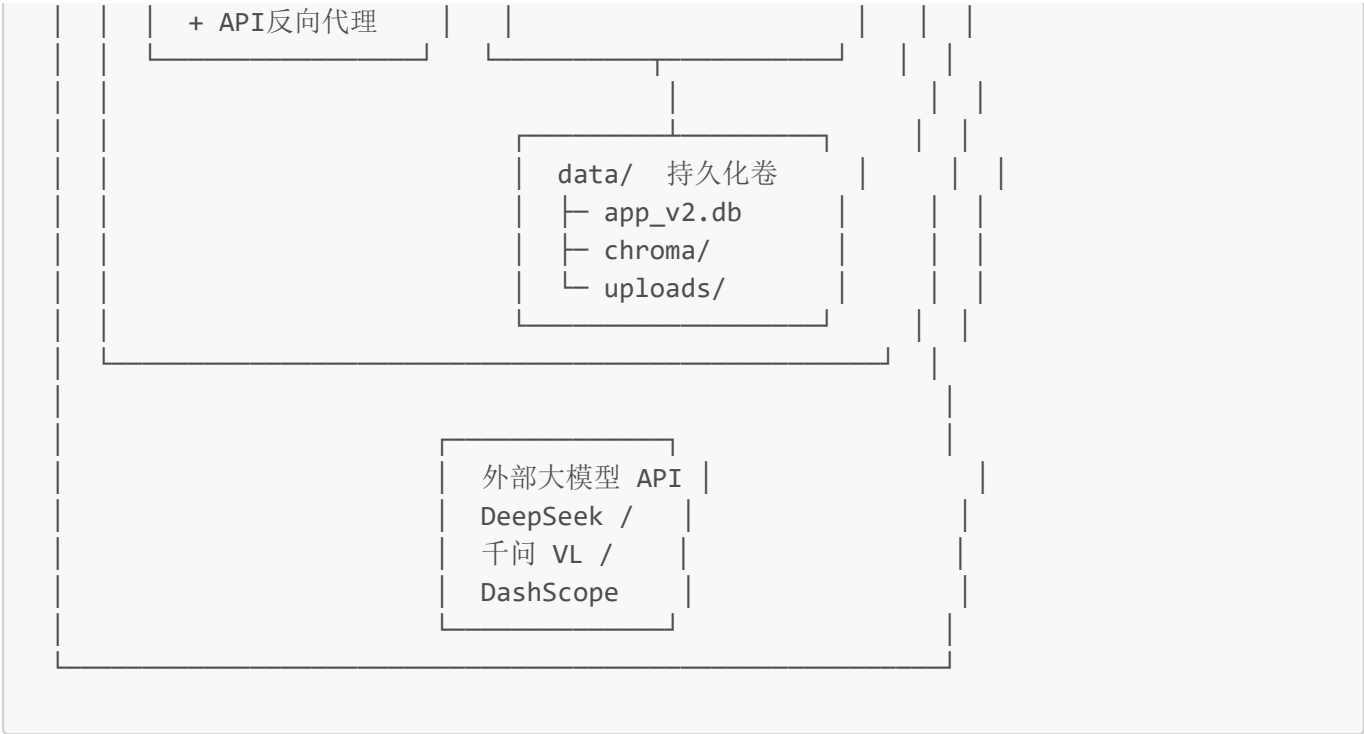
1.1 系统组成

"汇智海洋——设备检修知识检索与作业系统"由以下组件构成：

组件	技术栈	说明
前端	Vue 3 + Vite + Element Plus	PC Web 端，响应式适配。构建后为静态 HTML/JS/CSS 文件
后端	FastAPI (Python 3.10+)	REST API 服务，处理知识库、AI 对话、作业指引、审核流程、工单等业务逻辑
关系数据库	SQLite (开发/轻量) / PostgreSQL (生产)	存储用户、知识条目、对话记录、工单等业务数据
向量数据库	ChromaDB（嵌入式）	存储知识内容的向量索引，支持语义检索。以后端进程内嵌入方式运行
大模型 API	DeepSeek API / 阿里云 DashScope	外部服务，系统通过网络调用，无需本地 GPU
文件存储	本地文件系统（data/uploads/）	上传的图片、PDF、附件等文件落盘存储

1.2 部署架构图





1.3 容器编排说明

本系统采用 Docker Compose 进行多容器编排，包含两个核心服务：

服务名	容器名	端口映射	依赖
frontend	huizhi-frontend	80:80	等待 backend 健康后启动
backend	huizhi-backend	8000:8000	无（独立启动）

两个服务通过 **huizhi-net** 桥接网络通信。前端 Nginx 将 **/api/** 和 **/uploads/** 路径反向代理到后端。

设计原因（为什么不把前后端放在一个容器？）：

- 前后端可独立更新、独立扩缩容
- Nginx 作为前端容器可高效处理静态资源、Gzip 压缩、缓存
- 符合"Docker 一个容器一个职责"的最佳实践

2 环境要求与基础依赖安装

2.1 硬件与操作系统要求

项目	要求
CPU 架构	loongarch64（赛题强制要求）或 x86_64（开发验证）
操作系统	银河麒麟高级服务器版 V10 / V11
内核版本	Linux Kernel 4.19+（麒麟 V10 自带）
最小内存	4 GB（推荐 8 GB+，ChromaDB 向量检索需额外内存）
最小磁盘	20 GB（含系统、镜像、数据文件；推荐 50 GB+ 以便长期运行）

项目	要求
网络	需能访问互联网（调用大模型 API 必需）

验证系统信息：

```
# 查看架构
uname -m
# 预期输出：loongarch64

# 查看操作系统版本
cat /etc/os-release
# 预期输出：银河麒麟高级服务器操作系统 V10

# 查看内存
free -h

# 查看磁盘空间
df -h
```

2.2 Docker 与 Docker Compose 安装

****以下命令均在麒麟 V10 终端中执行。****每步解释命令含义，纯新手也能操作。

什么是 Docker? Docker 是一个容器化平台，将软件及其依赖打包到一个"容器"中运行，确保在不同环境中行为一致。本项目的后端和前端都打包在容器中，无需手动安装 Python 或 Node.js。

2.2.1 安装 Docker（麒麟 V10）

```
# 步骤1：更新软件包列表
sudo dnf update -y

# 步骤2：安装 Docker
sudo dnf install -y docker

# 步骤3：启动 Docker 服务
sudo systemctl start docker

# 步骤4：设置 Docker 开机自启动
sudo systemctl enable docker

# 步骤5：验证安装
docker --version
# 预期输出类似：Docker version 20.10.x
```

2.2.2 将当前用户加入 Docker 组（避免每次用 sudo）

```
# 将当前用户添加到 docker 组
sudo usermod -aG docker $USER

# 刷新组权限（或退出终端重新登录）
newgrp docker

# 验证：不加 sudo 直接运行
docker ps
```

2.2.3 安装 Docker Compose

```
# 步骤1: 下载 Docker Compose 二进制文件
# 注意: LoongArch 架构使用 loong64 版本
ARCH=$(uname -m)
if [ "$ARCH" = "loongarch64" ]; then
    COMPOSE_ARCH="loong64"
else
    COMPOSE_ARCH="x86_64"
fi

sudo curl -L "https://github.com/docker/compose/releases/download/v2.24.0/docker-
compose-linux-${COMPOSE_ARCH}" \
    -o /usr/local/bin/docker-compose

# 步骤2: 添加执行权限
sudo chmod +x /usr/local/bin/docker-compose

# 步骤3: 验证安装
docker-compose --version
# 预期输出类似: Docker Compose version v2.24.0
```

如果 GitHub 下载失败（网络限制），可在 Windows 上下载后通过 scp 或 U 盘传至虚拟机 `/usr/local/bin/docker-compose`，同样执行 `chmod +x`。

2.2.4 启动 Docker 服务

```
# 启动 Docker 守护进程
sudo systemctl start docker

# 检查 Docker 运行状态（应显示 active (running)）
sudo systemctl status docker
```

2.3 配置龙芯 Docker 镜像源（LoongArch 必需）

在 LoongArch 架构上，标准 Docker Hub 的 x86 镜像无法直接运行，需要从龙芯官方镜像仓库（cr.loongnix.cn）拉取适配 loongarch64 架构的镜像。

```
# 步骤1: 创建 Docker 配置文件目录
sudo mkdir -p /etc/docker

# 步骤2: 编辑 daemon.json (如文件不存在则创建)
sudo tee /etc/docker/daemon.json << 'EOF'
{
  "registry-mirrors": ["https://cr.loongnix.cn"]
}
EOF

# 步骤3: 重新加载配置并重启 Docker
sudo systemctl daemon-reload
sudo systemctl restart docker

# 步骤4: 验证配置
docker info | grep -A 5 "Registry Mirrors"
# 预期输出中应包含 https://cr.loongnix.cn
```

该步骤的作用：当执行 `docker pull` 或 `docker build` 时，Docker 优先从 `cr.loongnix.cn` 查找适配 loongarch64 的镜像（如 python、node、nginx），而不是 Docker Hub 的 x86 版本。

如果 `cr.loongnix.cn` 中没有所需的镜像版本，可回退方案见 [附录 8.4 龙芯架构适配说明](#)。

2.4 获取项目源码

```
# 方式1: 通过 git 克隆 (如果有 Git 仓库)
git clone <仓库地址> /opt/huizhi-ocean
cd /opt/huizhi-ocean

# 方式2: 通过 U 盘或 scp 将项目文件夹复制到虚拟机
# 假设复制到 /opt/huizhi-ocean
cd /opt/huizhi-ocean
```

项目目录确认：以下操作假设项目根目录为 `/opt/huizhi-ocean`，请根据实际情况调整路径。项目目录下应包含：

```
huizhi-ocean/
├── backend/          # 后端源码
│   ├── app/         # 应用代码
│   ├── .env         # 环境配置 (需要手动创建)
│   └── requirements.txt
├── frontend/        # 前端源码
├── docker/          # Docker 配置文件
│   ├── Dockerfile.backend
│   ├── Dockerfile.frontend
│   ├── nginx.conf
│   └── docker-compose.yml
```

```
| | .dockerignore
| | docs/          # 文档
```

3 项目配置文件准备

3.1 后端环境变量 (.env)

部署前需要在 `backend/` 目录下创建 `.env` 文件。此文件包含系统运行的关键配置。

```
cd /opt/huizhi-ocean
nano backend/.env
```

填入以下内容（根据实际情况修改 # `TODO` 标注的值）：

```
# ===== 应用基础配置 =====
APP_NAME=汇智海洋 - 设备检修知识检索与作业系统
APP_VERSION=0.1.0
DEBUG=false
SECRET_KEY=请替换为一串随机字符串（至少32位，用于JWT签名）
ACCESS_TOKEN_EXPIRE_MINUTES=480

# ===== 数据库配置 =====
# SQLite（轻量部署，推荐用于演示和小规模使用）
DATABASE_URL=sqlite+aiosqlite:///./data/app_v2.db

# PostgreSQL（生产环境取消下方注释，并注释 SQLite 行）
# DATABASE_URL=postgresql+asyncpg://huizhi:password@localhost:5432/huizhi_ocean

# ===== ChromaDB 向量数据库 =====
CHROMA_PERSIST_DIR=./data/chroma

# ===== 文件存储 =====
UPLOAD_DIR=./data/uploads
MAX_UPLOAD_SIZE_MB=50

# ===== DeepSeek 大模型配置 =====
# 文本主力模型 API（用于：AI 对话生成）
DEEPSEEK_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
DEEPSEEK_API_BASE=https://api.deepseek.com
DEEPSEEK_MODEL=deepseek-chat

# ===== 千问 VL 视觉模型配置 =====
# 用于：故障图片分析、PDF图片描述生成
QWEN_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
QWEN_API_BASE=https://dashscope.aliyuncs.com/compatible-mode/v1
QWEN_VL_MODEL=qwen-vl-max

# ===== Embedding 向量化模型 =====
```

```
# 选项：api（千问DashScope，推荐）| local（本地BGE模型）| builtin（ChromaDB内置）
EMBEDDING_PROVIDER=api
EMBEDDING_MODEL_NAME=BAAI/bge-small-zh-v1.5
EMBEDDING_DEVICE=cpu

# ===== CORS 跨域配置 =====
# 生产环境填写实际访问地址
CORS_ORIGINS=["*"]
```

配置项说明：

配置项	必填	说明
SECRET_KEY	✓	JWT 签名密钥。可使用 <code>openssl rand -hex 32</code> 生成随机字符串。生产环境必须修改！
DEEPSEEK_API_KEY	✓	DeepSeek API Key，从 <code>platform.deepseek.com</code> 获取
QWEN_API_KEY	✓	阿里云 DashScope API Key，从 <code>dashscope.aliyun.com</code> 获取
DATABASE_URL	✓	数据库连接字符串，默认 SQLite 无需修改
DEBUG	—	生产环境设为 <code>false</code> ，调试时可临时改为 <code>true</code>

API Key 也可通过管理后台动态配置：若 `.env` 中不填写 Key，管理员可在系统启动后通过"模型配置"页面挂载 API Key。详见操作手册第 5.9 节。

3.2 大模型 API 配置（数据库方式）

系统支持两种大模型配置方式：

- 1. **静态配置**（`.env` 文件）：在 `.env` 中填写 Key，系统启动时加载
- 2. **动态配置**（管理后台）：管理员通过"模型配置"页面在线配置，即时生效无需重启

两种方式的优先级：**数据库激活配置 > .env 静态配置**。

推荐：`.env` 中可以留空 Key（设为 `""`），部署后在管理后台动态配置，方便后续切换。

3.3 前端构建配置

前端构建通常无需额外配置。如需自定义后端 API 地址：

```
# 检查 frontend/vite.config.js 中的 proxy 配置
# Docker 部署时，前端 Nginx 已配置反向代理到 backend:8000
# 无需修改
```

4 使用 Docker Compose 一键部署

以下为**推荐部署方式**。所有命令在项目根目录 `/opt/huizhi-ocean` 下执行。

4.1 部署前检查清单

在部署前，逐项确认以下条件：

- ☐ 虚拟机架构为 loongarch64 (`uname -m`)
- ☐ Docker 已安装并运行 (`docker ps`)
- ☐ Docker Compose 已安装 (`docker-compose --version`)
- ☐ 龙芯镜像源已配置 (`docker info | grep cr.loongnix.cn`)
- ☐ 项目目录已完整放置在虚拟机 (`ls docker/ backend/ frontend/`)
- ☐ `backend/.env` 文件已创建并填写
- ☐ 端口 80 和 8000 未被占用 (`ss -tlnp | grep -E ':80|:8000'`)

4.2 构建镜像

```
cd /opt/huizhi-ocean

# 构建所有服务的镜像
# 首次构建约需 5-15 分钟（取决于网络和 CPU），后续构建会利用缓存，1-2 分钟
docker-compose -f docker/docker-compose.yml build

# 如果网络情况不佳，可尝试增加超时时间
# DOCKER_CLIENT_TIMEOUT=300 COMPOSE_HTTP_TIMEOUT=300 docker-compose -f
docker/docker-compose.yml build
```

构建过程解读（了解即可）：

- 构建前端镜像：**拉取 node:18 镜像 → npm install 安装依赖 → npm run build 生成静态文件 → 复制到 nginx 镜像
- 构建后端镜像：**拉取 python:3.10 镜像 → pip install 安装依赖 → 复制源码

如果构建成功，可看到两个镜像：

```
docker images | grep huizhi
# docker-huangizhi-frontend    latest    xxx MB
# docker-huangizhi-backend    latest    yyy MB
```

4.3 启动服务

```
# 后台启动所有服务
docker-compose -f docker/docker-compose.yml up -d

# 首次启动需要初始化数据库（约 10-30 秒）
# 查看启动状态
docker-compose -f docker/docker-compose.yml ps
```

预期输出（STATUS 列为 Up 即正常）：

NAME	STATUS
huizhi-backend	Up (healthy)
huizhi-frontend	Up

4.4 验证部署

检查后端：

```
# 在虚拟机内测试后端健康检查
curl http://localhost:8000/api/v1/health
# 预期输出: {"status":"ok","version":"0.1.0","server_time":"HH:MM:SS"}

# 测试登录接口
curl -X POST http://localhost:8000/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"admin123"}'
# 预期输出: {"access_token":"eyJ...", "user":{"..."}}
```

检查前端：

在浏览器中访问虚拟机 IP 地址：

`http://<虚拟机IP地址>`

应看到系统登录页面（紫色渐变背景，“汇智海洋”品牌面板）。

4.5 首次初始化操作

系统首次启动后，需要执行以下初始化操作：

1. 登录管理员账号：

默认管理员账号：

- 用户名：`admin`
- 密码：`admin123`

安全提醒：首次登录后，请在“个人工作台”中修改管理员密码，或通过管理后台“用户管理→重置密码”完成修改。

2. 配置大模型 API Key（如果 .env 中未配置）：

1. 进入管理后台 → 模型配置
2. 新增三个配置（文本、视觉、向量各一）：
 - 文本模型：`deepseek-chat`，API 地址 `https://api.deepseek.com`
 - 视觉模型：`qwen-vl-max`，API 地址 `https://dashscope.aliyuncs.com/compatible-mode/v1`

- 向量模型: `text-embedding-v4`, API 地址 <https://dashscope.aliyuncs.com/compatible-mode/v1>

3. 分别填写 API Key 并点击【激活】

3. 导入初始知识数据:

1. 进入管理后台 → 知识库管理
2. 使用【新增知识】手动编写, 或【导入 PDF】批量导入设备手册

4. 配置作业模板:

1. 进入管理后台 → 作业模板管理
2. 创建各设备型号的标准化检修模板

5 手动分步部署 (不使用 Docker)

如果 Docker 部署遇到困难 (如镜像源不可用), 可采用直接在麒麟 V10 上安装依赖的传统部署方式。此方式不需要 Docker, 但需要手动安装 Python 和 Node.js。

5.1 Python 环境安装

```
# 麒麟 V10 通常自带 Python 3, 确认版本
python3 --version
# 需 >= 3.10, 若版本不足需升级

# 安装 pip 和编译工具
sudo dnf install -y python3-pip python3-devel gcc gcc-c++ make

# 升级 pip
python3 -m pip install --upgrade pip
```

5.2 Node.js 环境安装

```
# 方式1: 通过 dnf 安装 (推荐)
sudo dnf install -y nodejs npm

# 方式2: 如果 dnf 源的版本过低 (< 18), 从龙芯社区下载
# 访问 https://loongnix.cn 下载 loongarch64 版本的 Node.js 18+ 二进制包
# 解压到指定目录并配置环境变量

# 验证
node --version # 预期 >= 18
npm --version
```

5.3 后端部署

```
cd /opt/huizhi-ocean/backend

# 步骤1: 创建虚拟环境
python3 -m venv venv
source venv/bin/activate

# 步骤2: 安装依赖
pip install -r requirements.txt
# LoongArch 上部分包含 C 扩展的包可能需要从源码编译
# 若失败可尝试: pip install --no-binary :all: -r requirements.txt

# 步骤3: 确保 .env 文件存在并已配置

# 步骤4: 创建数据目录
mkdir -p data data/uploads data/chroma

# 步骤5: 启动后端（开发模式）
uvicorn app.main:app --host 0.0.0.0 --port 8000

# 步骤6（生产模式，后台运行）：
nohup uvicorn app.main:app --host 0.0.0.0 --port 8000 > backend.log 2>&1 &
# 进程 PID 会显示，可用 kill <PID> 停止
```

验证后端：

```
curl http://localhost:8000/api/v1/health
```

5.4 前端部署

```
cd /opt/huizhi-ocean/frontend

# 步骤1: 安装依赖（使用国内镜像加速）
npm install --registry=https://registry.npmmirror.com

# 步骤2: 构建生产版本
npm run build
# 构建产物在 dist/ 目录下

# 步骤3: 将 dist/ 复制到 Nginx 的静态文件目录
sudo cp -r dist/* /usr/share/nginx/html/
```

5.5 Nginx 配置

```
# 步骤1: 安装 Nginx
sudo dnf install -y nginx
```

```
# 步骤2: 创建配置文件
sudo nano /etc/nginx/conf.d/huizhi-ocean.conf
```

填入以下内容:

```
server {
    listen 80;
    server_name localhost;

    root /usr/share/nginx/html;
    index index.html;

    gzip on;
    gzip_min_length 1024;
    gzip_types text/plain text/css application/json application/javascript
text/xml;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_buffering off;
        proxy_read_timeout 120s;
    }

    location /uploads/ {
        proxy_pass http://127.0.0.1:8000;
    }
}
```

```
# 步骤3: 测试配置
sudo nginx -t

# 步骤4: 启动 Nginx
sudo systemctl start nginx
sudo systemctl enable nginx

# 步骤5: 配置防火墙
sudo firewall-cmd --add-port=80/tcp --permanent
sudo firewall-cmd --reload
```

6 运维管理

6.1 常用 Docker 命令

操作	命令	说明
查看运行中的容器	<code>docker ps</code>	列出所有运行中的容器及基本信息
查看所有容器	<code>docker ps -a</code>	包含已停止的容器
启动服务	<code>docker-compose -f docker/docker-compose.yml up -d</code>	后台启动
停止服务	<code>docker-compose -f docker/docker-compose.yml stop</code>	停止但不删除容器
停止并删除	<code>docker-compose -f docker/docker-compose.yml down</code>	停止并删除容器、网络
重启服务	<code>docker-compose -f docker/docker-compose.yml restart</code>	重新启动所有容器
查看后端日志	<code>docker logs huizhi-backend</code>	查看后端容器的控制台输出
查看前端日志	<code>docker logs huizhi-frontend</code>	查看前端容器的控制台输出
进入后端容器	<code>docker exec -it huizhi-backend bash</code>	进入容器内部执行命令

执行所有 `docker-compose` 命令前，需先 `cd /opt/huizhi-ocean`（项目根目录）。

6.2 查看日志

```
# 实时查看后端日志（最近100行，持续输出）
docker logs --tail 100 -f huizhi-backend

# 实时查看前端日志
docker logs --tail 100 -f huizhi-frontend

# 查看近 5 分钟的日志
docker logs --since 5m huizhi-backend

# 查看所有服务的日志
docker-compose -f docker/docker-compose.yml logs --tail 50
```

6.3 重启与更新

更新代码后重新部署：

```
cd /opt/huizhi-ocean
```

```
# 步骤1: 拉取最新代码 (Git) 或替换源码文件 (U盘)

# 步骤2: 重新构建镜像
docker-compose -f docker/docker-compose.yml build --no-cache

# 步骤3: 重启服务
docker-compose -f docker/docker-compose.yml down
docker-compose -f docker/docker-compose.yml up -d

# 步骤4: 验证
docker-compose -f docker/docker-compose.yml ps
curl http://localhost:8000/api/v1/health
```

仅更新后端代码:

```
# 如果只修改了 backend/ 目录下的代码
docker-compose -f docker/docker-compose.yml up -d --no-deps --build backend
```

6.4 数据备份

系统中的数据保存在 Docker 命名卷 `docker_backend-data` 中, 或直接挂载到 `backend/data/` 目录。

备份数据库和上传文件:

```
# 在虚拟机上执行

# 创建备份目录 (按日期命名)
BACKUP_DIR="/opt/backups/huizhi-ocean/$(date +%Y%m%d_%H%M%S)"
mkdir -p "$BACKUP_DIR"

# 方式1: 如果使用 Docker 卷
docker cp huizhi-backend:/app/data "$BACKUP_DIR/data"

# 方式2: 如果数据保存在 backend/data/ (推荐)
cp -r /opt/huizhi-ocean/backend/data "$BACKUP_DIR/data"

# 同时备份 .env 配置文件
cp /opt/huizhi-ocean/backend/.env "$BACKUP_DIR/"

echo "备份完成: $BACKUP_DIR"
```

设置定期自动备份 (crontab):

```
# 编辑 crontab
crontab -e

# 添加每天凌晨 3 点自动备份
```

```
0 3 * * * cp -r /opt/huizhi-ocean/backend/data /opt/backups/huizhi-ocean/$(date +\%Y\%m\%d)/
```

6.5 健康监控

健康检查接口：

```
# 后端健康检查（本地）
curl http://localhost:8000/api/v1/health

# 后端健康检查（通过前端代理）
curl http://localhost/api/v1/health
```

Docker 自动健康检查：

- `docker-compose.yml` 已配置 Docker HEALTHCHECK
- 后端每 30 秒自动检查一次 `/api/v1/health`
- 前端每 30 秒自动检查一次 `/`
- 状态可通过 `docker ps` 的 STATUS 列查看
- 如果连续 3 次检查失败，容器状态变为 `unhealthy`
- 配合 `restart: unless-stopped`，容器异常退出时自动重启

检查服务状态：

```
# 查看容器健康状态
docker ps --format "table {{.Names}}\t{{.Status}}"

# 查看容器资源占用
docker stats --no-stream
```

7 故障排除

7.1 常见启动问题

症状	可能原因	解决方法
<code>docker-compose up</code> 失败， 端口占用	80 或 8000 端口被其他 进程占用	<code>ss -tlnp grep -E ':80 :8000'</code> 查找占用 进程，kill 后重试
容器反复重启	应用程序启动失败（检 查日志）	<code>docker logs huizhi-backend</code> 查看错误信息
<code>curl</code> 返回 Connection Refused	后端未启动完成	等待 30 秒，待启动完成后再试
健康检查失败	后端耗时初始化超时	增加 HEALTHCHECK 的 start_period 值

7.2 LoongArch 特定问题

症状	诊断	解决方法
<code>docker pull</code> 镜像 Not Found	x86 镜像无法在 LoongArch 上运行	确认已配置 <code>cr.loongnix.cn</code> 镜像源 (2.3 节)
<code>pip install</code> 某个包失败	该包的 C 扩展无预编译版，需要源码编译	安装编译工具： <code>sudo dnf install -y gcc gcc-c++ python3-devel</code> ，然后重试 <code>pip install</code>
<code>chromadb</code> 安装失败	<code>onnxruntime</code> 无 LoongArch 版本	见 8.4 节 ChromaDB 适配说明
<code>pymupdf</code> (fitz) 安装失败	MuPDF 库无 LoongArch 预编译	<code>pip install --no-binary pymupdf pymupdf</code> 从源码编译

7.3 数据库锁定问题

症状：所有请求返回 500 错误，日志显示 `database is locked`

原因：SQLite 在并发写入时可能出现锁定；进程异常崩溃后残留 `-journal` 文件导致下次启动阻塞。

解决方法：

```
# 步骤1: 进入 data 目录
cd /opt/huizhi-ocean/backend/data

# 步骤2: 检查是否有残留 journal 文件
ls -la *.db-journal

# 步骤3: 如果存在，停止后端后删除
docker-compose -f docker/docker-compose.yml stop backend
rm -f *.db-journal

# 步骤4: 重启
docker-compose -f docker/docker-compose.yml up -d backend
```

后端启动时已内置 journal 文件自动清理逻辑。如果仍遇到锁定问题，可按上述步骤手动处理。

7.4 AI 服务不可用

症状：AI 对话返回"AI 服务暂时不可用"

可能原因：

- 1. **API Key 未配置：**检查 .env 中的 `DEEPSEEK_API_KEY` 和 `QWEN_API_KEY`，或在管理后台"模型配置"中激活
- 2. **API 额度耗尽：**大模型 API 余额不足，登录 API 平台查看
- 3. **网络不通：**虚拟机无法访问外网

诊断命令：

```
# 在虚拟机中测试能否访问 DeepSeek API
curl -I https://api.deepseek.com

# 测试 DNS 解析
nslookup api.deepseek.com
```

7.5 端口冲突

```
# 查看端口占用情况
ss -tlnp | grep -E ':80|:8080'

# 如果 80 端口被占用（如已有的 Nginx 或 Apache）
# 方案1: 修改 docker-compose.yml 中 frontend 的端口映射
# 将 "80:80" 改为 "8080:80", 然后通过 http://IP:8080 访问

# 方案2: 停止占用端口的服务
sudo systemctl stop nginx # 如果 Nginx 占用
sudo systemctl stop httpd  # 如果 Apache 占用
```

8 附录

8.1 目录结构

```
huizhi-ocean/
├── backend/
│   ├── app/
│   │   └── api/
│   └── (等)
│       ├── core/
│       ├── models/
│       ├── schemas/
│       └── services/
│   ├── data/
│   │   ├── app_v2.db
│   │   ├── chroma/
│   │   └── uploads/
│   ├── .env
│   └── requirements.txt
├── frontend/
│   └── src/
│       ├── views/user/
│       ├── views/admin/
│       ├── components/
│       ├── composables/
│       ├── api/
│       ├── router/
│       └── stores/

# 项目根目录
# 后端源码
# 应用代码
# API 路由 (auth, chat, knowledge, tickets
# 核心配置 (config, database, security)
# 数据库 ORM 模型
# Pydantic 数据校验
# 业务逻辑 (LLM, 向量库, 文件服务)
# 运行时数据 (持久化)
# SQLite 数据库文件
# ChromaDB 向量存储
# 用户上传文件
# 环境变量配置
# Python 依赖清单
# 前端源码
# 用户端页面
# 管理端页面
# 通用组件
# 组合式逻辑 (useChat等)
# API 调用封装
# 路由配置
# Pinia 状态管理
```

		package.json	# Node.js 依赖清单
		vite.config.js	# 构建配置
		docker/	# Docker 配置文件
		Dockerfile.backend	# 后端镜像定义
		Dockerfile.frontend	# 前端镜像定义
		nginx.conf	# Nginx 配置
		docker-compose.yml	# 多容器编排
		.dockerignore	# 构建排除文件
		docs/	# 文档
		软件操作手册.md	

8.2 端口清单

端口	服务	用途
80	Nginx（前端容器）	对用户提供 Web 页面 + API 反向代理
8000	Uvicorn（后端容器）	后端 REST API 服务
5432	PostgreSQL（可选）	关系数据库（仅切换到 PG 时使用）

8.3 Docker 命令速查表

命令	用途
docker ps	查看运行中的容器
docker ps -a	查看所有容器（含已停止）
docker images	查看本地镜像列表
docker logs <容器名>	查看容器日志
docker exec -it <容器名> bash	进入容器 shell
docker restart <容器名>	重启单个容器
docker rm <容器名>	删除已停止的容器
docker rmi <镜像名>	删除镜像
docker system prune -a	清理不用的容器、镜像、网络（慎用！）
docker-compose up -d	后台启动 Compose 所有服务
docker-compose down	停止并删除 Compose 所有服务
docker-compose restart	重启 Compose 所有服务
docker-compose build --no-cache	重新构建镜像（不使用缓存）

8.4 龙芯架构适配说明

8.4.1 核心挑战

LoongArch (loongarch64) 与 x86_64 是两种完全不同的 CPU 指令集架构。x86_64 的二进制文件（包括 Docker 镜像、.so 动态库、Python C 扩展）无法在 LoongArch 上直接运行。因此：

- Docker 基础镜像必须使用 loongarch64 版本（cr.loongnix.cn 提供）
- 带有 C 扩展的 Python 包需要在 LoongArch 上**从源码编译**（`pip install --no-binary`）
- Node.js 的 native addon (如 node-gyp 编译的包) 需要重新编译

8.4.2 依赖兼容性预估

依赖包	LoongArch 兼容	说明
fastapi / uvicorn / pydantic	✓	纯 Python，跨架构通用
sqlalchemy / aiosqlite	✓	纯 Python
httpx / aiofiles	✓	纯 Python
langchain / langchain-community	✓	纯 Python（依赖链较长，但均为纯 Python）
chromadb	⚠	依赖 onnxruntime，可能需要从源码编译或降级使用
cryptography / bcrypt	⚠	有 C 扩展，需编译环境
pymupdf (fitz)	⚠	需要从源码编译 MuPDF 库
Pillow	⚠	C 扩展，麒麟 V10 通常有 loongarch64 wheel
python-docx / openpyxl	✓	纯 Python
pdfplumber	✓	纯 Python（备选 PDF 解析）
apscheduler	✓	纯 Python

8.4.3 ChromaDB 替代方案

如果 ChromaDB 在 LoongArch 上始终无法安装（onnxruntime 兼容性问题），可考虑以下替代方案：

方案 A：使用内置 Embedding

ChromaDB 默认的 `all-MiniLM-L6-v2` 模型也是 ONNX 格式，可能同样不兼容。建议坚持使用 API 模式（千问 DashScope Embedding），这是纯 HTTP 调用，与架构无关。

```
# .env 中设置（这是项目的默认设置）
EMBEDDING_PROVIDER=api
```

方案 B：替换为 FAISS

如果 ChromaDB 本身也无法运行，可用 FAISS + SQLite 替代向量检索：

```
# FAISS 是 Meta 开发的向量检索库，纯 CPU 运行，不依赖 ONNX  
pip install faiss-cpu
```

方案 C：纯关键词检索降级

作为最后备选，系统可在无向量数据库的情况下运行（搜索降级为数据库 LIKE 查询 + 大模型直接回答）。此时 AI 对话的"检修模式"退化为直接问答。

8.4.4 获取帮助

- **龙芯开源社区**：<https://loongnix.cn> — 获取 LoongArch 适配的软件包
- **龙芯镜像仓库**：<https://cr.loongnix.cn> — LoongArch Docker 镜像
- **项目仓库**：通过客服工单联系开发团队获取 LoongArch 适配支持

文档版本：V1.0 | 编制日期：2026-06-18 | 编制人：汇智海洋项目组

© 2026 汇智海洋 · 设备检修知识检索与作业系统